ACADNET LINUX CHEAT SHEET

Olympiad open-book reference · AcadNet 9-10 · Calculatoare & Sisteme de Operare · All commands from 2023/2024/2025 exams covered

[HOW TO SEARCH] Ctrl+F in your PDF reader. Every section is clearly titled. Commands are in GREEN · Flags/variants in BLUE · Tips in ORANGE · Warnings in RED · Notes in PURPLE. Exam problem references like 'Prob 2025 Q2b' tell you exactly which question uses that command.

◆ FILES & DIRECTORIES

■ Creating Directories

`mkdir dirname`	create a single directory
`mkdir -p a/b/c`	create nested dirs (no error if exists) — the -p flag is your best friend
`mkdir -p {calc,is,rl}/{9-10,11-12}`	brace expansion: creates 6 dirs at once
`mkdir -p AcadNet/{calc/{9-10,11-12},is/{9-10,11-12},rl/{9-10,11-12}}`	exact structure from AcadNet 2025 Prob 1a

■ Creating Files

`touch file.txt`	create empty file or update timestamp
`touch file_{1..50}`	create 50 files: file_1, file_2 … file_50
`fallocate -l 12M subiect_1`	instantly create a 12 MB file (fastest method)
`for i in $(seq 1 50); do fallocate -l 12M subiect_$i; done`	50 files of 12MB each — AcadNet 2025 Prob 1b
`dd if=/dev/urandom of=file bs=1M count=10`	10 MB file filled with random data
`dd if=/dev/zero of=file bs=1M count=10`	10 MB file filled with zeros (faster)
`echo 'text' > file.txt`	create file with content (overwrites)
`echo 'text' >> file.txt`	append to file (does NOT overwrite)

[TIP] fallocate is near-instant; dd is slow but creates real random data. Use fallocate for size tasks, dd for random-data tasks.

■ Listing & Viewing

`ls -lh`	long listing with human-readable sizes (KB, MB)
`ls -lhS`	sort by file size, largest first
`ls -v`	natural sort (1,2,3…10,11 instead of 1,10,11,2)
`ls --sort=version`	same as -v; use this for subiect_1…subiect_50
`ls -la`	show hidden files (starting with .) too
`ls -lh | sort -k5 -h`	pipe to sort by size column human-readable
`tree`	display full directory tree (install: apt install tree)
`tree --sort=version`	tree with numeric sort on filenames
`tree -h`	tree showing file sizes
`cat file.txt`	print file contents to screen
`less file.txt`	paginated view (q to quit, / to search)
`head -n 10 file.txt`	first 10 lines of file
`tail -n 10 file.txt`	last 10 lines of file
`tail -f logfile`	follow file in real time (great for logs)

◆ ARCHIVES (zip & tar)

■ ZIP Archives

`zip archive.zip file1 file2`	create zip with specific files
`zip -r archive.zip dir/`	zip entire directory recursively
`zip -r archive.zip .`	zip current directory
`unzip archive.zip`	extract zip archive here
`unzip -l archive.zip`	LIST zip contents without extracting — Prob 1d
`unzip archive.zip -d /target/`	extract to specific directory

■ TAR Archives (.tar.gz)

`tar -czf archive.tar.gz dir/`	CREATE compressed tar.gz from directory
`tar -czf archive.tar.gz Makefile *.c *.h`	create with specific files/extensions
`tar -tvf archive.tar.gz`	LIST contents without extracting — Prob 5b
`tar -tf archive.tar.gz`	same as -tvf but compact output
`tar -xzf archive.tar.gz`	EXTRACT tar.gz here
`tar -xzf archive.tar.gz -C /target/`	extract to specific directory
`tar -czf db.tar.gz database/`	archive entire database dir — Prob 5b
`tar -czf arch.tar.gz -T filelist.txt`	create from a list of files
`tar -rf archive.tar newfile`	append file to existing tar (no -z)

[TIP] tar flags memory trick: c=Create, x=eXtract, t=lisT, z=gZip, f=File, v=Verbose

■ Common Archive Patterns

`tar -czf acadnet_rules.tar.gz Makefile *.c *.h`	Makefile pack rule — Prob 4c
`zip -r hierarchy.zip . && unzip -l hierarchy.zip`	zip + immediately verify contents
`tar -czf backup_$(date +%F).tar.gz dir/`	archive with date in filename

■ Copying, Moving, Deleting

`cp src dst`	copy file
`cp -r src/ dst/`	copy directory recursively
`cp -u src dst`	copy only if src is newer than dst
`cp -rp src/ dst/`	copy preserving permissions and timestamps
`mv file newname`	rename file or move it
`mv *.txt dir/`	move all .txt files into dir/
`rm file`	delete file (no recycle bin!)
`rm -rf dir/`	delete directory and all contents (be careful)
`rm -i file`	interactive delete — asks confirmation

[WARN] DANGER: rm -rf has no undo. Always double-check the path. Never run as root unless certain.

■ Finding Files

`find . -name '*.txt'`	find all .txt files from current dir
`find /var/log -name '*.log'`	all .log files under /var/log
`find . -mmin -10`	files modified in last 10 minutes
`find . -mmin -5`	files modified in last 5 minutes
`find . -type f`	only files (not directories)
`find . -type d`	only directories
`find . -size +5M`	files larger than 5 MB
`find . -name '*.log' -exec cp {} /tmp/ \;`	find + run command on each result
`find . -name '*.txt' | xargs ls -lh`	find + pass results to another command

■ Disk Usage & Size

`du -sh .`	total size of current directory (human-readable)
`du -sh *`	size of each item in current directory
`du -ah`	size of every single file recursively
`du -sh /var/log`	total size of /var/log
`df -h`	disk space of all mounted filesystems

◆ PERMISSIONS & OWNERSHIP

■ chmod — Change File Permissions

`chmod 777 dir/`	rwxrwxrwx — full access for everyone (public)
`chmod 770 dir/`	rwxrwx--- — owner+group full, others nothing (secret)
`chmod 755 file`	rwxr-xr-x — owner full, others read+exec
`chmod 700 dir/`	rwx------ — only owner can access
`chmod 644 file`	rw-r--r-- — owner rw, others read only
`chmod 600 file`	rw------- — only owner can read/write (config)
`chmod 400 file`	r-------- — read-only even for owner
`chmod -R 777 dir/`	apply recursively to all files inside
`chmod a-w file`	remove write from ALL users
`chmod g+x file`	add execute for group
`chmod o-r file`	remove read from others

[TIP] Permissions: r=4, w=2, x=1. Add them: rwx=7, rw-=6, r-x=5, r--=4. Three digits = owner|group|others

■ chown — Change Owner/Group

`chown user file`	change file owner
`chown user:group file`	change owner AND group
`chown -R dbowner:dbadmin secret/`	recursive ownership — Prob 5d
`chgrp dbadmin dir/`	change only the group
`ls -la`	verify ownership (shows user group)

■ Users & Groups

`useradd -m username`	create user with home directory
`useradd -m -G groupname username`	create user and add to group
`usermod -aG groupname username`	add EXISTING user to group (-a = append!)
`groupadd groupname`	create a new group
`passwd username`	set or change a user's password
`usermod -L username`	LOCK user (cannot login)
`usermod -s /sbin/nologin user`	disable login shell (alternative lock)
`chage -M 90 username`	password expires after 90 days — Prob 2023
`id username`	show UID, GID, groups of a user
`groups username`	list groups a user belongs to
`cat /etc/passwd | grep user`	verify user exists in system
`cat /etc/group | grep grpname`	verify group and its members
`who`	show logged-in users
`w`	show users + what they're running

[WARN] usermod -aG: the -a (append) flag is CRITICAL. Without it, you replace all groups instead of adding!

◆ PROCESSES

■ Viewing Processes

`ps aux`	show ALL processes from ALL users
`ps aux | grep firefox`	find a specific process by name
`ps aux --sort=-%mem`	sort by memory usage, highest first
`ps aux --sort=-%mem | head -6`	top 5 processes by memory (line 1 is header)
`ps aux --sort=-%cpu | head -6`	top 5 by CPU usage
`ps -eo pid,ppid,user,cmd`	custom columns: PID, parent PID, user, command
`ps -u root -o pid,ppid,user,cmd`	all root processes with custom cols — Prob 2024
`ps --ppid 1 -o pid`	all processes whose parent is PID 1
`pstree -p`	show process tree with PIDs
`pstree -u`	show process tree with usernames
`echo $$`	PID of the CURRENT shell — always useful
`top`	interactive process monitor (q to quit)
`htop`	better interactive monitor (if installed)

■ Killing & Signaling Processes

`kill PID`	send SIGTERM (graceful stop) to process
`kill -9 PID`	send SIGKILL (force kill, cannot be caught)
`kill -SIGKILL PID`	same as kill -9, explicit name
`killall sleep`	kill all processes named 'sleep'
`killall -9 sleep`	force kill all 'sleep' processes
`pkill -f 'pattern'`	kill processes matching a pattern in cmdline
`jobs -p | xargs kill -9`	kill all background jobs in current shell

■ Background & Job Control

◆ SCHEDULING (at & cron)

■ at — Schedule One-Time Commands

`at 14:30`	schedule interactive job at 14:30 today
`at 00:00 08/01/2025`	schedule at midnight on Aug 1 2025 — Prob 2025 Q2b
`echo 'command' | at TIME`	pipe command directly into at
`echo 'echo "Hello AcadNet"' | at 00:00 08/01/2025`	exact solution for Prob 2025 Q2b
`atq`	list pending at jobs
`atrm JOB_ID`	remove a pending at job

■ cron — Schedule Recurring Commands

`crontab -e`	edit current user's crontab
`crontab -l`	list current crontab entries
`crontab -r`	remove all crontab entries (careful!)
`0 0 1 8 * /path/cmd`	run at 00:00 on August 1st every year
`* * * * * cmd`	run every minute
`0 * * * * cmd`	run every hour at :00
`@reboot cmd`	run once at system boot
`*/5 * * * * cmd`	run every 5 minutes

[TIP] Cron format: MIN HOUR DAY MONTH WEEKDAY command. Weekday: 0=Sun…6=Sat

`command &`	run command in background
`sleep 30 &`	background sleep process
`for i in {1..10}; do sleep 30 & done`	start 10 sleep processes in background
`jobs`	list all background jobs in current shell
`jobs -p`	list only PIDs of background jobs
`fg %1`	bring job #1 to foreground
`bg %1`	resume stopped job #1 in background
`Ctrl+Z`	suspend (pause) current foreground process
`Ctrl+C`	terminate current foreground process
`nohup command &`	run in background, survives logout
`disown %1`	detach job from shell (keeps running after logout)

■ renice — Change Process Priority

`renice 15 -p PID`	set nice value to 15 for one process
`for pid in $(ps aux --sort=-%mem | awk 'NR>1&&NR;<=6{print $2}'); do renice 15 -p $pid; done`	renice top 5 by memory — Prob 2025 Qa
`nice -n 10 command`	start command with nice value 10

[TIP] Nice values: -20 = highest priority, 0 = default, 19 = lowest priority. Only root can set negative values.

■ Named Processes — Advanced

`exec -a 'custom name' sleep 1000 &`	start sleep but /proc shows 'custom name' — Prob 2025 Q2d
`ps aux | grep 'de ce rinocerul'`	verify custom-named process is running
`cat /proc/PID/cmdline | tr '\0' ' '`	check actual cmdline of a process
`pgrep -a sleep`	find sleep processes with their PIDs

◆ GIT

■ Setup & Init

`git config --global user.name 'student'`	set global username — Prob 2025 Q7a
`git config --global user.email 'student@acadnet.ro'`	set global email — Prob 2025 Q7a
`git init`	initialize a directory as a git repo
`git clone URL`	clone a remote repository locally
`git clone URL dir/`	clone into a specific directory

■ Daily Workflow

`git status`	show staged, unstaged, and untracked files
`git add filename`	stage a specific file
`git add .`	stage ALL changes in current directory
`git add -A`	stage all changes including deletions
`git commit -m 'message'`	commit staged changes with a message
`git log --oneline`	compact commit history (one line each)
`git log`	full commit history with dates and diffs
`git diff`	show unstaged changes
`git diff --staged`	show staged but not yet committed changes
`git show HEAD`	show the last commit's changes

■ Branches

`git branch`	list all local branches
`git branch -a`	list all branches including remote
`git branch networking`	create branch named 'networking'
`git checkout -b networking`	create AND switch to branch 'networking' — Prob 2025 Q7c
`git checkout main`	switch to main branch
`git merge branchname`	merge branch into current branch

■ Editing History — EXAM CRITICAL

`git commit --amend -m 'new message'`	change the message of the LAST commit — Prob 2025 Q7d
`git commit --amend --no-edit`	amend last commit keeping same message
`git rebase -i HEAD~3`	interactive rebase: edit last 3 commits
`git reset --soft HEAD~1`	undo last commit, keep changes staged
`git reset --hard HEAD~1`	undo last commit AND discard changes
`git stash`	temporarily save uncommitted work
`git stash pop`	restore stashed work

[WARN] git commit --amend only modifies the LAST commit. For older commits use git rebase -i HEAD~N then change 'pick' to 'reword'

■ Practical Git Patterns

`cp /etc/passwd . && git add passwd && git commit -m 'add passwd'`	copy file + add to git — Prob 2025 Q7b
`ifconfig > ports && git add ports && git commit -m 'Suspicious ports'`	save ifconfig output + commit — Prob 2025 Q7c
`git log --oneline | head -5`	show 5 most recent commits

◆ TERMINAL CUSTOMIZATION (PS1)

■ ANSI Color Codes

`\[\e[0m\]`	RESET — always add after colored text!
`\[\e[1;31m\]`	BOLD RED — use for username
`\[\e[1;32m\]`	BOLD GREEN — use for @ symbol or hostname
`\[\e[1;33m\]`	BOLD YELLOW — use for hostname
`\[\e[35m\]`	MAGENTA/PURPLE — use for path
`\[\e[1;34m\]`	BOLD BLUE
`\[\e[1;36m\]`	BOLD CYAN
`\[\e[47m\]`	WHITE BACKGROUND
`\[\e[43m\]`	ORANGE BACKGROUND

[WARN] ALWAYS wrap color codes in \[...\] in PS1. Without them, bash miscalculates line length and breaks line editing!

■ PS1 Special Variables

`\u`	current username
`\h`	hostname (short, up to first dot)
`\H`	full hostname
`\w`	current directory (full path)
`\W`	current directory (basename only)
`\$`	$ for normal user, # for root
`\$?`	exit code of LAST command — use in PROMPT_COMMAND
`\t`	current time HH:MM:SS
`\d`	current date
`\n`	newline in prompt

■ PS1 Examples — Exam Patterns

◆ MAKEFILE

■ How Make Works

`make`	run the first/default rule
`make build`	run the 'build' rule
`make run`	run the 'run' rule
`make clean`	run the 'clean' rule
`make pack`	run the 'pack' rule
`make -n`	dry run: show what would be executed
`make -f MyMakefile`	use a specific Makefile name

[WARN] CRITICAL: Makefile recipe lines MUST be indented with a real TAB character, not spaces. Editors often auto-convert — be careful!

■ Makefile Structure — Prob 4a

`.PHONY: build run clean pack`	declare non-file targets (prevents conflicts)
`build:`	rule with no dependencies — always runs
`gcc -o acadnet2025 main.c firststep.c`	compile two source files — Prob 4a
`./acadnet2025`	run the binary right after build
`run: acadnet2025`	run depends on binary existing
`./acadnet2025`	execute the binary
`clean:`	cleanup rule
`rm -f acadnet2025 *.o`	delete executables and .o files (not zeus.o)

■ Makefile with Object Files — Prob 4b

`export PS1='\[\e[1;31m\]\u\[\e[0m\]@\[\e[1;33m\]\h\[\e[0m\]:\w\$ '`	user=RED, @=white, host=YELLOW — Prob 2025 Q3a
`export PS1='\[\e[1;31m\]\u\[\e[0m\]\[\e[1;32m\]@\[\e[0m\]\[\e[1;33m\]\h\[\e[0m\]:\w\$ '`	user=red, @=GREEN, host=yellow — Prob 2025 Q3a exact
`export PS1='\[\e[1;31m\]\u\[\e[0m\]@\[\e[1;32m\]\h\[\e[0m\]:\[\e[35m\]\w\[\e[0m\]\$ '`	user=red, host=green, path=purple — AcadNet 2023
`export PS1='\u@$(hostname -I | awk "{print \$1}"):\w\$ '`	show IP instead of hostname — Prob 2025 Q3c
`export PS1='\u@\h:\w \[\e[43m\]\[\e[30m\]$?\[\e[0m\]\$ '`	show exit code on orange bg — AcadNet 2023

■ Making PS1 Persistent

`echo 'export PS1="..."' >> ~/.bashrc`	add to ~/.bashrc for persistence
`source ~/.bashrc`	reload .bashrc in current session
`exec bash`	restart shell to test .bashrc changes
`echo 'export PS1="..."' >> /etc/bash.bashrc`	make PS1 default for ALL users

■ Cursor & cowsay

`printf '\e[5 q'`	blinking underline cursor — Prob 2025 Q3d
`echo -e '\e[5 q'`	alternative cursor command
`printf '\e[1 q'`	blinking block cursor
`printf '\e[2 q'`	steady block cursor
`echo 'printf "\e[5 q"' >> ~/.bashrc`	make cursor persistent
`cowsay 'ACADNET 2025'`	cow says message — Prob 2025 Q3b
`cowsay -f hellokitty 'ACADNET 2025'`	Hello Kitty says message — Prob 2025 Q3b
`cowsay -l`	list all available cow characters
`cowthink 'message'`	thought bubble variant
`fortune | cowsay`	classic combo: random fortune + cow

`acadnet2025: zeus.o secondstep.o`	binary depends on two .o files
`gcc -o acadnet2025 zeus.o secondstep.o`	link object files into binary
`zeus.o:`	compile zeus.c to object (if needed)
`gcc -c zeus.c -o zeus.o`	compile source to object file
`clean: rm -f acadnet2025 *.o`	clean but note: won't delete zeus.o if excluded

■ Makefile Pack Rule — Prob 4c

`pack:`	create archive without deleting pack
`tar -czf acadnet_rules.tar.gz Makefile *.c *.h`	archive Makefile + all .c and .h files
`$(wildcard *.c)`	get all .c files dynamically
`$(patsubst %.c,%.o,$(SRC))`	convert .c list to .o list

[TIP] The 'pack' target must NOT delete itself. Use .PHONY: pack and never add rm pack to it.

◆ SYSTEMD & BOOT ANALYSIS

■ Boot Time Analysis — Prob 2025 Q6

`systemd-analyze`	show total boot time: firmware+loader+kernel+userspace
`systemd-analyze blame`	show each service and how long it took (unsorted)
`systemd-analyze blame | sort -k1 -h`	sort services by time, ASCENDING — Prob 2025 Q6b
`systemd-analyze blame | sort -k1 -hr`	sort DESCENDING (slowest first)
`systemd-analyze blame | sort -k1 -h | awk '{print $2}'`	show ONLY service names sorted — Prob 2025 Q6c
`systemd-analyze blame | sort -k1 -h | awk '{print $2}' | uniq`	deduplicate same-time services — Prob 2025 Q6c
`systemd-analyze critical-chain`	show the critical boot chain
`systemd-analyze plot > boot.svg`	generate a visual SVG of boot sequence

■ Service Management

`systemctl status service`	check if service is running
`systemctl start service`	start a service now
`systemctl stop service`	stop a service now
`systemctl restart service`	stop then start a service
`systemctl enable service`	start service at every boot
`systemctl disable service`	don't start at boot
`systemctl list-units --type=service`	list all active services
`journalctl -u service`	view logs for a specific service
`journalctl -b`	all logs from current boot
`journalctl -xe`	recent logs with explanations (great for debugging)
`journalctl -f`	follow logs in real time

◆ NETWORKING

■ Viewing IP Addresses

`ip a`	show all network interfaces and IP addresses
`ip a show eth0`	show only the eth0 interface
`hostname -I`	print all IP addresses of this machine (clean)
`hostname -I | awk '{print $1}'`	first IP only, no mask — simplest method
`ip a | grep -oP '(?<=inet )\d+\.\d+\.\d+\.\d+'`	extract IP with regex (no mask) — Prob 2025 Q3c
`ifconfig`	classic tool (install net-tools if missing)
`ifconfig > ports`	save ifconfig output to file — Prob 2025 Q7c
`ip route`	show routing table and default gateway

■ SSH — Remote Access

`ssh user@IP`	connect to remote machine
`ssh student@192.168.1.1`	example connection — AcadNet 2023 Prob 4a
`ssh-keygen -t rsa`	generate RSA key pair (~/.ssh/id_rsa)
`ssh-copy-id user@IP`	copy public key to server (enables no-password login)
`cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys`	manually add key for passwordless SSH
`scp file.txt user@IP:/path/`	copy file TO remote machine
`scp user@IP:/path/file.txt .`	copy file FROM remote machine
`scp -r dir/ user@IP:/path/`	copy entire directory to remote

■ Network Diagnostics

`ping -c 4 8.8.8.8`	test connectivity (4 packets)
`curl wttr.in/Bucuresti`	weather for Bucharest in terminal — AcadNet 2023 Q1a
`curl wttr.in/$1`	weather for city passed as script argument — Q1b
`curl -s wttr.in/?format=3`	compact one-line weather
`netstat -tuln`	show open/listening ports
`ss -tuln`	modern alternative to netstat
`nslookup domain.com`	DNS lookup
`dig domain.com`	detailed DNS lookup

◆ TEXT PROCESSING & REGEX

■ grep — Search in Files

`grep 'pattern' file`	search for pattern in file
`grep -i 'pattern' file`	case-insensitive search
`grep -r 'pattern' dir/`	recursive search in directory
`grep -n 'pattern' file`	show line numbers
`grep -c 'pattern' file`	count matching lines
`grep -v 'pattern' file`	show lines NOT matching (invert)
`grep -E 'regex' file`	use extended regex (ERE)
`grep -oP 'regex' file`	show ONLY the matched part (Perl regex)
`grep -l 'pattern' *.txt`	show only filenames that match
`grep -c 'c' /etc/shadow`	count occurrences of 'c' — Prob 2024 Q3c
`cat /etc/shadow | grep -o 'c' | wc -l`	count character occurrences (alternative)

■ Email Regex — Prob 2024 Q6

`grep -E '^[a-zA-Z][a-zA-Z0-9._]*@[a-zA-Z]+\.[a-z]{2,4}$' emails.txt`	valid email filter — Prob 2024 Q6a
`grep -E '.*\.com$' valid.txt > valid_emails.txt`	filter only .com emails — Prob 2024 Q6b
`grep -cE '^[a-zA-Z]...' emails.txt`	count valid emails

■ sed — Stream Editor (Find & Replace)

`sed 's/old/new/' file`	replace FIRST occurrence per line
`sed 's/old/new/g' file`	replace ALL occurrences (global)
`sed 's/old/new/g' file > newfile`	save result to new file
`sed -i 's/old/new/g' file`	edit file IN-PLACE (modifies original)
`sed 's/@[^.]*\./@christmasparty./' f`	replace email domain — Prob 2024 Q6c
`sed -n '5,10p' file`	print only lines 5 to 10
`sed '/pattern/d' file`	delete lines matching pattern

■ awk — Column/Field Processing

`awk '{print $1}' file`	print first column (space-delimited)
`awk '{print $2}' file`	print second column
`awk 'NR>1&&NR;<=6{print $2}' file`	print col 2 for rows 2 to 6
`awk -F: '{print $1}' /etc/passwd`	use : as delimiter, print field 1
`ps aux --sort=-%mem | awk 'NR>1&&NR;<=6{print $2}'`	get PIDs of top 5 memory processes

■ sort, uniq, wc, cut

`sort file`	sort lines alphabetically
`sort -n file`	sort numerically
`sort -h file`	sort human-readable sizes (1K < 1M < 1G)
`sort -r file`	reverse sort
`sort -k1 -h`	sort by first column, human-readable
`sort | uniq`	remove duplicate lines (must sort first)
`sort | uniq -c`	count occurrences of each unique line
`wc -l file`	count lines
`wc -c file`	count characters/bytes
`wc -w file`	count words
`cut -d: -f1 /etc/passwd`	extract field 1, using : as delimiter

■ Other Text Utilities

`yes 'Ac@dnet$' | head -2024 > out.txt`	repeat line 2024 times — Prob 2024 Q3b
`printf 'line\n%.0s' {1..2024} > out.txt`	alternative: printf loop
`tr 'a-z' 'A-Z' < file`	convert lowercase to uppercase
`tr -d '\n' < file`	remove all newlines
`tee file.txt`	write to file AND show on screen simultaneously
`xargs`	build commands from stdin (use with find/grep)

◆ HARDWARE INFO — Prob 2024 Q9

■ CPU & Cache

`lscpu`	full CPU information: cores, threads, cache sizes
`lscpu | grep 'L[123] cache'`	show L1, L2, L3 cache sizes — Prob 2024 Q9a
`lscpu | grep -i aes`	check for AES hardware acceleration — Prob 2024 Q9b
`lscpu | grep -i 'cpu(s)'`	number of CPUs/cores
`nproc`	number of available CPU cores
`cat /proc/cpuinfo`	detailed per-core CPU information

■ GPU & Network Cards

◆ PIPING, REDIRECTION & ONE-LINERS

■ Redirection — Input/Output

`command > file`	redirect stdout to file (OVERWRITES)
`command >> file`	redirect stdout to file (APPENDS)
`command 2> file`	redirect STDERR to file
`command 2>&1`	redirect stderr to same place as stdout
`command > file 2>&1`	redirect BOTH stdout and stderr to file
`command < file`	use file as stdin input
`command << 'EOF'`	heredoc: type multi-line input until EOF
`command > /dev/null`	discard output completely
`command > /dev/null 2>&1`	discard ALL output (silent)

[WARN] > always overwrites. >> always appends. Never use > when you mean >>!

`lspci | grep -i vga`	list video/GPU cards — Prob 2024 Q9c
`lspci | grep -i display`	alternative GPU detection
`lspci -k | grep -A2 -i ethernet`	network card + driver used — Prob 2024 Q9d
`lshw -short`	short hardware summary (all devices)
`lshw -class network`	network hardware details
`dmidecode -t memory`	RAM details (type, speed, slots)
`free -h`	current RAM and swap usage (human-readable)
`free -k`	RAM and swap in kilobytes — Prob 2024 Q3d
`vmstat`	system memory, CPU, I/O statistics

■ System Info

`uname -a`	full kernel and OS information
`uname -r`	kernel version only
`hostnamectl`	hostname, OS, kernel summary
`cat /etc/os-release`	Linux distribution details
`uptime`	how long the system has been running
`last reboot`	history of system reboots

◆ SUDOERS — Restricting Commands

■ Allow Group to Run Only ls — AcadNet 2023 Q2e

■ Pipes — Chaining Commands

`cmd1 | cmd2`	pipe: stdout of cmd1 becomes stdin of cmd2
`ps aux | grep firefox`	find firefox process
`ps aux | sort | head -20`	sorted processes, first 20
`cat file | grep pattern | wc -l`	count matching lines — multi-stage pipe
`ls -1 | wc -l`	count files in directory
`find . -name '*.log' | xargs rm`	find + delete all .log files
`find . -name '*.txt' | xargs grep 'word'`	search for word inside all .txt files
`du -ah | sort -h | tail -20`	20 largest files sorted by size
`history | grep 'git'`	search your command history for git commands
`cat /etc/passwd | cut -d: -f1 | sort`	list all usernames sorted

■ tee — Split Output

`command | tee file.txt`	print to screen AND save to file
`command | tee -a file.txt`	print to screen AND APPEND to file
`ifconfig | tee network_info.txt`	save and view network info

■ Command Chaining

`cmd1 && cmd2`	run cmd2 ONLY if cmd1 succeeds (exit code 0)
`cmd1 || cmd2`	run cmd2 ONLY if cmd1 FAILS
`cmd1 ; cmd2`	run cmd2 regardless of cmd1 result
`git add . && git commit -m 'msg'`	only commit if add succeeded
`make build && ./acadnet2025`	only run if build succeeded
`ping -c1 host || echo 'offline'`	print 'offline' if ping fails

`visudo`	safely edit /etc/sudoers (use this, not nano!)
`%gods ALL=(ALL) /bin/ls`	group gods can ONLY run ls with sudo
`%gods ALL=(ALL) NOPASSWD: /bin/ls`	same, but no password prompt
`sudo -l -U username`	list what sudo commands user can run
`sudo -u username command`	run command AS another user
`%groupname ALL=(ALL) /cmd1, /cmd2`	allow multiple commands
`username ALL=(ALL) ALL`	give a user full sudo (admin)

[WARN] % prefix means GROUP in sudoers. Without %, it's a username. Always use visudo — syntax errors can lock you out!

■ xargs — Build Command Arguments

`find . -name '*.txt' | xargs rm`	delete all found .txt files
`find . -name '*.log' | xargs gzip`	compress all .log files
`cat list.txt | xargs mkdir`	create a directory for each line
`jobs -p | xargs kill -9`	kill all background jobs — Prob 2025 Q2c
`echo 'hello world' | xargs -n1 echo`	print each word on separate line

■ Useful One-Liners from Exams

`for i in {1..10}; do sleep 30 & done; sleep 1; kill $(pgrep -n sleep)`	10 sleep processes + SIGKILL — Prob 2025 Q2c
`for i in {1..10}; do sleep 30 & done; jobs -p | xargs kill -9`	alternative: kill via jobs
`exec -a 'de ce rinocerul are corn?' sleep 1000 &`	background sleep with custom name — Prob 2025 Q2d
`yes 'Ac@dnet$' | head -2024 > acad_out.txt`	file with line repeated 2024× — Prob 2024 Q3b
`for i in $(seq 1 50); do fallocate -l 12M subiect_$i; done`	50 × 12MB files — Prob 2025 Q1b
`find . -name '*.txt' -exec rename 's/^\.?/\./' {} \;`	hide .txt files (add dot prefix)
`diff dir1/ dir2/ | grep 'Only in dir2' | awk '{print $4}'`	find files only in dir2
`echo $(( ($(date -d '25 Dec' +%j) - $(date +%j) + 365) % 365 ))`	days until Christmas — AcadNet 2023 Q1c

◆ BATMAN vs JOKER — Prob 2025 Q8

■ Goal: Make id always show root — no aliases, no direct binary edit

`which id`	find where the real 'id' binary is (/usr/bin/id)
`echo $PATH`	see current PATH — order determines command priority
`mkdir -p ~/bin`	create personal bin directory
`export PATH=~/bin:$PATH`	prepend ~/bin to PATH (shell now finds your id first)
`gcc -o ~/bin/id fakeid.c`	compile a fake 'id' binary in ~/bin
`echo 'export PATH=~/bin:$PATH' >> /etc/bash.bashrc`	make it apply to ALL users (persistent)
`echo 'export PATH=~/bin:$PATH' >> /etc/environment`	system-wide PATH override
`hash -r`	clear shell command cache after PATH change
`type id`	verify which 'id' command would run now

[TIP] Fake id.c content: main(){system("/usr/bin/id --user=root 2>/dev/null || /usr/bin/id -u root");} — compile to ~/bin/id and prepend ~/bin to PATH in /etc/bash.bashrc

◆ ENVIRONMENT & SHELL

■ Variables

`export VAR=value`	set environment variable (available to child processes)
`VAR=value`	set shell variable (local to current shell only)
`echo $VAR`	print value of variable
`echo $HOME`	home directory path
`echo $PATH`	command search path
`echo $USER`	current username
`echo $SHELL`	current shell path
`env`	list all environment variables
`printenv VAR`	print specific env variable
`unset VAR`	remove a variable
`export TZ='date +%m/%d/%y'`	TZ trick: executing it shows date — AcadNet 2023 Q6b

■ Startup Files — Making Things Persistent

`~/.bashrc`	runs for every new interactive bash shell
`~/.bash_profile`	runs at login shell only
`/etc/bash.bashrc`	system-wide bashrc for ALL users
`/etc/environment`	system-wide env vars (not shell-specific)
`/etc/profile`	system-wide profile (login shells)
`source ~/.bashrc`	apply changes without opening new terminal
`echo 'command' >> ~/.bashrc`	run command every time terminal opens
`echo 'export VAR=val' >> ~/.bashrc`	persist variable across sessions

[TIP] For changes visible to ALL users: use /etc/bash.bashrc or /etc/environment. For yourself only: ~/.bashrc

■ Showing RAM & Processes at Terminal Start — AcadNet 2023 Q6a

`echo 'free -h' >> ~/.bashrc`	show RAM every time terminal opens
`echo 'ps aux | wc -l' >> ~/.bashrc`	show process count at startup
`echo 'echo "RAM:"; free -h; echo "Processes: $(ps aux | wc -l)"' >> ~/.bashrc`	combined startup message

◆ SCRIPTING ESSENTIALS

■ Script Basics

`#!/bin/bash`	shebang — always first line of bash script
`chmod +x script.sh`	make script executable
`./script.sh`	run script in current directory
`bash script.sh`	run script explicitly with bash
`bash -x script.sh`	debug mode — shows each command before running

■ Variables & Arguments in Scripts

`$1, $2, $3`	positional arguments passed to script
`$#`	number of arguments
`$@`	all arguments as separate words
`$0`	the script name itself
`$?`	exit code of last command (0=success)
`name="John"`	assign variable (no spaces around =)
`echo ${name}`	use variable (braces for clarity)
`result=$(command)`	capture command output into variable
`readonly VAR=val`	constant (cannot be changed)

■ Conditionals

`if [ -f file ]; then echo 'yes'; fi`	check if file exists
`if [ -d dir ]; then ...; fi`	check if directory exists
`if [ $? -eq 0 ]; then ...; fi`	check if last command succeeded
`[ -z "$var" ]`	true if variable is empty
`[ -n "$var" ]`	true if variable is NOT empty
`[ "$a" = "$b" ]`	string equality
`[ $a -eq $b ]`	numeric equality
`[ $a -gt $b ]`	numeric greater than

■ Loops

`for i in {1..10}; do echo $i; done`	loop 1 to 10
`for i in $(seq 1 50); do ...; done`	loop 1 to 50 (seq is more flexible)
`for f in *.txt; do echo $f; done`	loop over all .txt files
`while [ condition ]; do ...; done`	while loop
`for i in {1..1000}; do mkdir -p acadnet/$i/{1..10}; done`	mass directory creation — AcadNet 2023 Q3a
■ Weather Script — AcadNet 2023 Q1	
`#!/bin/bash`	first line
`curl -s wttr.in/Bucuresti`	Prob 1a: hardcoded Bucharest weather
`curl -s wttr.in/$1`	Prob 1b: city as argument ($1)
`days=$(( ($(date -d '25 Dec' +%j) - $(date +%j) + 365) % 365 ))`	calc days to Christmas — Q1c
`echo "Au mai ramas $days zile pana vine Mosu!"`	output message — Q1c
■ Useful Shortcuts	
`Ctrl+R`	reverse search through command history
`Ctrl+A / Ctrl+E`	go to beginning / end of line
`Ctrl+U`	delete from cursor to beginning of line
`Ctrl+K`	delete from cursor to end of line
`Ctrl+L`	clear screen (same as 'clear')
`!!`	repeat last command
`!$`	last argument of previous command
`!git`	repeat last command starting with 'git'
`history | grep cmd`	search command history
`Alt+.`	paste last argument of previous command